

Article 1 - Yifei Fang

Y. Xu, "MUARF: Leveraging Multi-Agent Workflows for Automated Code Refactoring," 2025 *IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, Ottawa, ON, Canada, 2025, pp. 226-227, doi: 10.1109/ICSE-Companion66252.2025.00071.

This paper presents MUARF, a tool that uses LLMs to automatically improve and clean up code in the way a human developer would. It looks at examples of good refactoring, breaks the task into steps like a team of developers, and uses extra tools to check that the updated code still works correctly and looks clean. The results show it can usually produce working, high-quality code that feels similar to what a human would write. This matters today because modern software is very large and hard to maintain, and developers spend a lot of time fixing and improving existing code. Tools like MUARF could save time, reduce errors, and make maintaining software much easier.

Article 2 - Yifei Fang

J. Cordeiro, S. Noei and Y. Zou, "LLM-Driven Code Refactoring: Opportunities and Limitations," 2025 *IEEE/ACM Second IDE Workshop (IDE)*, Ottawa, ON, Canada, 2025, pp. 32-36, doi: 10.1109/IDE66625.2025.00011.

This paper explains that while AI tools like LLM are getting pretty good at automatically improving code, they still have an important weakness. They can make mistakes or hallucinate changes that accidentally break how the software works. Refactoring is supposed to make code cleaner without changing what it does, but current AI systems aren't always reliable enough to be trusted on their own, so developers still need to double-check their work. The paper looks at these problems and suggests ways researchers could make AI refactoring safer and more dependable. This matters today because more developers are starting to rely on AI tools inside their coding environments, and making these tools more accurate and trustworthy is key if we want them to truly save time instead of creating new bugs.

Article 3 - Kwesi Michai

J. Cordeiro, S. Noei, Y. Zou, "An Empirical Study on the Code Refactoring Capability of Large Language Models," *ACM Transactions on Software Engineering and Methodology*, 2024. Available: <https://dl.acm.org/doi/10.1145/3801158>.

This paper studies how well large language models (LLMs) can perform code refactoring compared to human developers. The authors evaluate different models on real-world Java projects, focusing on their ability to remove code smells, maintain functional correctness, and improve overall code quality. The results show that LLMs perform well on simple and repetitive refactoring tasks, often producing consistent improvements. However, they are less effective when handling complex changes that require deeper reasoning and understanding of the codebase. I consider this article relevant because it highlights the growing role of AI in software engineering and shows how LLMs can already assist developers in maintaining and improving code, making development more efficient.

Article 4 - Kwesi Michai

A. Ampatzoglou et al., "AI-assisted code refactoring: Where can it be helpful and where do humans outperform it?," *Information and Software Technology*, 2026. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0164121226000956>.

This paper explores when AI tools are useful for code refactoring and when human developers still perform better. The authors conducted a controlled experiment with developers completing refactoring tasks both with and without AI assistance, focusing on applying design patterns and SOLID principles. The results show that AI-assisted approaches often produce more correct implementations of specific coding practices, especially when the task is clearly defined. However, human developers outperform AI in the earlier stages of refactoring, such as identifying problems, understanding the system, and reasoning about trade-offs in code quality.

I consider this article relevant because it clearly shows that AI is most effective as a support tool rather than a replacement for developers. It highlights that while AI can improve accuracy and speed in structured tasks, human insight is still necessary for deeper thinking and decision-making, which is essential for maintaining high-quality software.